A digital twin that can be checked against measurements

M13 • 26 Oct 2027 - 21 Nov 2027 • 75 hours

Monthly outcome

Build a quadruped model in Gazebo, verify it against simple physical examples and connect the controller in stages: first SIL, then PIL and single-leg HIL with a real processor and communication channel. Deliver a digital twin with reports on fidelity, timing and HIL testing. The quadruped is an intermediate architecture on the way to a six-legged robot: physical assembly is scheduled for M22, and the transition to six legs for the conditional M23 stage.

A digital twin must reproduce a specific geometry and use parameters and measurements from the actual system. Start with the leg CAD model, actuator specifications and M7-M10 logs. Identify the sources of mass, inertia, damping, limits, delays and noise. An unmeasured parameter may be assumed if it is explicitly labelled and given a range for sensitivity checks. A calculated curve must never be presented as an experimental measurement.

Prerequisites

From M12, you need Xacro, joint names, hardware interfaces, launch files and diagnostics. From earlier stages, you need controller firmware, the communication protocol, a secured single-leg or joint test rig, and rules for limiting applied commands. Before HIL, understand every packet field and the meaning of its timestamps. The model can be developed without an actuator; checking hardware and actual delays requires a real test rig.

The main environment is ROS 2 Jazzy, Ubuntu 24.04 and Gazebo Harmonic with gz_ros2_control. ARM64 installation is supported, but verify the particular plugins, graphics, GPU-processed sensors, virtual networking and device access separately. Running a server without a GUI and rendering depth images without a window are different modes: the latter still requires a working rendering engine. Any replacement of missing depth data with ideal values must be explicitly declared.

What the evidence establishes

Check contact, noise and timing quantitatively. Successful simulation does not establish actual strength, heating, surface friction or emergency stopping. HIL validates operation only within the declared hardware and software configuration; the physical actuator and foot contact remain separate experiments when represented by models. The G2 HIL criterion remains open until the corresponding experiment in this module is completed, even though the checkpoint itself is scheduled earlier.

Organising the 75 hours

Four work blocks

Block A, hours 1-20: SDF, physics and simple reference cases. Block B, 21-40: the quadruped, sensors and tests without a graphical interface. Block C, 41-60: SIL and PIL, scenarios and timing boundaries. Block D, 61-75: single-leg HIL and comparison with measurements. Each of the first three blocks contains 10 hours of theory and analysis and 10 hours of practice; the fourth contains 5 hours of analysis and 10 hours of practice. Total: 35 + 40 = 75 hours. The work steps on pages 9-10 allocate these same hours; do not add them again.

Divide each ten-hour theory allocation into five two-hour sessions: 20 minutes to reconstruct the previous model; 45 minutes to read the assigned section; 40 minutes to derive a small example or analyse a log; and 15 minutes to record the conclusion. Divide each block's ten-hour Sunday practical into five two-hour work periods with breaks between them. Breaks do not count towards net study hours. Split the final five analysis hours as 2 + 2 + 1.

Start with a question, then choose the tool

The main tools are Gazebo Harmonic, ROS 2 Jazzy and gz_ros2_control; Python for scenarios and analysis; and the existing microcontroller and physical communication channel for HIL. Begin each session with a testable question: how should the input affect the output, which state is unobservable, and what data can establish this? Before running the experiment, record the expected sign, scale or event sequence. Afterwards, compare the prediction with the result and investigate at least one discrepancy; producing a plot alone does not complete the task.

Maintain one repository with model, config, experiments, tests and reports directories. Every experiment needs a unique identifier, code version, configuration, parameter sources, initial state, units and raw data. The analysis script must read a saved experiment and generate its report without manual corrections to measurements. The monthly deliverable is a Gazebo digital twin with fidelity and HIL reports; every exercise adds to its verification suite.

Time accounting and reducing scope

Calendar boundaries may overlap with the neighbouring module. Rest, holiday periods and integration checks remain as specified in the overall roadmap; place the numbered steps in available study intervals without counting hours twice. No additional hours are assigned to this module. Assess equipment and library readiness at the start, rather than spending the practical on an unplanned platform change.

If you fall behind, reduce visual mesh detail and the number of worlds, and omit the second physics platform. Retain analytical reference cases, contact verification, the four-leg structure, a sensor reference scene and a dynamic single-leg experiment. Separate SIL, PIL and HIL results and a table of discrepancies against measurements are mandatory. Do not add gait generation or RL yet. If the target processor or physical rig is unavailable, leave that item open and describe its verification procedure; another run on the host computer does not replace it.

Verify the physics model before adding the robot

Study: the world and the solver

Study SDF world/model/link/joint/sensor, the physics step, gravity, contact and friction coefficients. Identify the selected physics engine and its available parameters; tag names do not guarantee identical interpretation across engines. Mass, inertia and collision geometry determine dynamics, while visual defines appearance. A detailed visual foot may differ from a simple contact geometry, provided the contact model is explained.

Add joint limits, damping and an actuator model. Ideal torque control, ideal position control and an actuator with an internal control loop and delay produce different results. First test a simple mass and a pendulum: their simulations can be compared with analytical calculations. Choose the physics step by considering convergence and computational cost. A smaller step will not correct an erroneous mass, negative inertia or incorrect axis direction.

Task 1. Analytical reference cases

Create a freely falling body without contact and a small-angle pendulum. For free fall, compare with $z(t)=z0+v0t-gt^2/2$ before impact; for the small-angle pendulum, compare its period with $2\pi\sqrt{(l/g)}$, provided the model is a point mass on a massless link. Repeat with steps dt and dt/2. Deliver error curves and a table showing how the metric changes when the step is reduced. Check units of m, s and kg, identical initial conditions and a fixed duration. Error check: if pendulum inertia does not match the chosen formula, correct the reference calculation before adjusting solver settings.

Task 2. Foot contact and an inclined plane

Use a simple block with the mass and contact geometry of a foot. Place it on a horizontal surface, then vary the inclination. Record penetration, sliding velocity, normal force and energy change. Compare the onset of sliding with the teaching approximation $\tan(\alpha)\approx\mu$ for the idealised model; explain the effects of the solver, damping and contact shape. Deliver a scenario series with tolerances chosen before the final comparison. Check that a block with zero friction cannot remain stationary on an incline without another cause; any qualitative change when the step is reduced needs an explanation.

The block deliverable is a set of simple cases with expected physical behaviour and at least one reproducible contact-model failure. Only then add dozens of links and sensors.

The quadruped model and sensors

Study: repeatable geometry

Assemble a body and four instances of the verified leg using Xacro. Joint names and axes must match M12; mirroring the left and right sides requires a separate sign check. Before exporting to simulation, verify total mass, centre of mass, joint origin/axis, permitted travel, collision geometry and transmission ratio. Connect command and state interfaces through gz_ros2_control following the Jazzy/Harmonic conventions. Declaring a transmission does not by itself guarantee that the plugin applies its reduction ratio.

Add an IMU, joint-state and foot-contact sensors, and a depth sensor. For each, specify its coordinate frame, units, update rate, noise, delay and ground-truth source. The simulator publishes ground truth on a separate stream for evaluation; the controller receives only permitted sensor readings. Ground-truth leakage distorts the assessment of algorithm accuracy. Check IMU tilt using the adopted gravity convention, and depth against a plane at a known distance.

Task 3. Turn one leg into four

1) Verify one leg in three known configurations. 2) Compare the link endpoint position with an independent kinematic calculation. 3) Create the remaining instances with their mirroring parameters. 4) Command the same intended movement on all legs and compare directions. Save Xacro, SDF, launch files, a table of joints and interfaces, and a geometry test. Check the mass sum against the parts list, physically meaningful inertias and that the body does not fall through the floor. If a mirrored leg moves in the wrong direction, the test must identify the specific joint.

Task 4. Sensor reference scenes without a GUI

Create a world with a horizontal platform, a wall at a known distance and a fixed robot position and orientation. Run the server without a GUI, wait for the streams and record IMU, contacts, joint states and depth. Prepare an automatic test with a timeout and a report on frequencies, noise and distance error. A disabled sensor, unavailable rendering engine or empty depth map must produce an explicit error or be recorded as a limitation. Do not fill a missing stream with zeros. Compare ground truth and readings at aligned timestamps; model delay separately from publication frequency.

For the arm, transfer only the architectural principle: parameterised links, joint limits, correct contact geometry and separate sensor-reading and ground-truth streams. A complete arm and tactile models are outside this month's scope.

SIL and PIL: where each component runs

Study: experiment boundaries

In SIL, the controller runs on the host computer and controls a mechanical model in a closed loop. In PIL, its code runs on the target processor and exchanges data with the model; this checks the build, numerical types and calculations on the selected hardware. PIL can run step by step, slower than real time. HIL in the next block also checks the physical communication channel and the declared closed-loop timing requirements. Terminology varies between tools, so every report diagram must show the actual distribution of components.

For each scenario, specify the random-number seed, versions, initial robot configuration and controller state, world parameters, physics step, noise model, injected fault and expected result. A fixed seed improves repeatability but does not guarantee bitwise equality for a multithreaded physics engine across operating systems. Distinguish simulation time (sim time) from real elapsed time (wall time). Real-time factor = sim time increment / wall time increment; this ratio does not directly describe whether an individual packet arrives on time.

Task 5. SIL with the same controller

Use the M10/M11 controller library without copying its equations into a new node. 1) Run a fixed single-leg trajectory. 2) Record error, commands, saturation and contact states. 3) Repeat three times with the same seed. 4) Introduce sensor delay and additional friction. Deliver a scenario catalogue and discrepancy report. Check that identical inputs produce results within the declared numerical tolerance and that the fault scenario survives a restart. If callback order affects the result, save seq and fix the data-input order in the test.

Task 6. PIL on the microcontroller

Build the computational core for a real target processor, such as the board from earlier modules. The host sends a snapshot of the leg state; the processor calculates a command and returns seq/status. In stepwise mode, the model continues only after the specified response. Save the protocol, firmware hash and a target-versus-host comparison of commands for identical inputs. Differences between float and double precision must remain within a tolerance justified in advance; reject NaN and overflow. A damaged packet or missing seq must not result in data being accepted as a new valid snapshot.

After comparing functionality, measure target computation time and round-trip time separately. Do not subtract readings from unsynchronised clocks: measure the round trip on one side and local computation with the processor timer. A PIL result does not yet establish timely control of a physical actuator.

Single-leg HIL and comparison with real mechanics

Study: HIL requires real hardware

The minimum HIL experiment in this programme uses a real controller with firmware and a physical USB/UART/CAN channel. They exchange state and command data with the leg model in a closed loop under timing requirements defined in advance. The controller acts on the model, whose response returns to the controller. If all computation runs on a laptop, the experiment is SIL. If the board merely receives and returns a packet without calculating control or affecting the model, the experiment tests communication but does not yet demonstrate HIL control.

Declare the period, permitted data age, late-data handling and stopping method in advance. Simulation on the host must meet the chosen timing mode; if performance is insufficient, record failure of the timing criterion. A slow stepwise run is useful for PIL but does not satisfy timely HIL. Choose the threshold from your loop's timing budget, rather than a universal number in an instruction sheet.

Task 7. Close the hardware loop for one leg

Use the verified PIL protocol and connect the firmware to the leg model through a physical channel. 1) Run a small, safe trajectory with bounded commands. 2) Record sim time, host receive/send, target compute duration and seq. 3) Introduce dropped packets, delay and a disconnected line. 4) Check the watchdog, fallback mode and resumption through an explicitly defined state. Prepare an HIL report, firmware, component diagram and fault log. Every cycle must correspond unambiguously to a state snapshot; reject expired commands. The actuator and contact are modelled here, so this experiment does not establish physical stopping of the leg.

Task 8. Connect the model to a physical test rig

On a secured real leg or joint, repeat one previously authorised small M10 test and compare position, delay, settling time and available load information with the model. Use the same command and consistent initial conditions. Estimate parameters from one set of recordings and validate on another. Deliver a simulation-versus-measurement table and a list of discrepancies. Check that you do not fit a single damping value to align every curve while concealing an incorrect delay or internal actuator-loop model. If the physical rig is unavailable, prepare the procedure and leave the corresponding validation open.

Use the command limits and mounting arrangements from the already verified mechanics. Do not connect a real actuator to a virtual environment in a way that lets two controllers set the same command simultaneously. Before the experiment, explicitly select model-only, HIL-controller or physical-bench mode.

Digital-twin specification and scenario catalogue

Project structure

Store geometry and interfaces in robot_description. The simulation directory contains SDF worlds, physics and sensor settings, and the bridge; scenarios contains initial states, inputs and criteria; firmware contains the target-processor build; hil_bridge contains the protocol and timeouts; fidelity contains model and test-rig data; reports contains comparisons and limitations. Record the Git revision, Jazzy/Harmonic versions, engine, operating system and architecture, physics step, seed and actuator model in the common manifest.

The mandatory world catalogue contains an analytical reference case, foot contact, a stationary quadruped with sensors and a single-leg test scene. A complete gait is not needed yet. The quadruped model must have correct axes, limits and sensor streams, while the dynamic experiment may use one leg with a fixed body. This reduces the number of simultaneously unknown causes of discrepancy.

Accuracy requirements by subsystem

For geometry, compare lengths and coordinates in known configurations; for mass and inertia, compare CAD data and available measurements; for the actuator, compare delay and transient response; for contact, compare sliding and penetration. Check IMU noise, frequency and delay; depth-sensor distance and pixel dropout; and communication losses, data age and missed deadlines. Each table row must give the parameter source, criterion, result, limitations and next physical experiment.

Do not assign one number such as "digital-twin accuracy 95%". Position error in millimetres and timing error in milliseconds describe different properties. Set tolerances according to the question the model will answer: interface testing, step-size selection, controller tuning or contact. A model may be suitable for watchdog testing and unsuitable for predicting foot slip.

Automating a run

The launch script must open the world, wait for components to become ready, reset the state, execute the scenario, record streams, check criteria and terminate every process. Account separately for launch failure, missing clock or sensor, and timeout. Run tests without a GUI first; check rendering availability separately for depth. Save a machine-readable pass/fail result and a plot that makes its cause understandable.

Define the HIL boundary in every report: which signals are physical, which are generated, where the control law runs and how time is measured. A photograph of the board and wiring helps reproduction, but the evidence consists of commands, responses and the closed-loop model's reaction to an active hardware controller.

Testing the model under faults

Physics and sensors

F1 - Incorrect inertia or axis. Change one entry and check a known configuration or a small input. Numerical geometry or dynamics checks must detect the discrepancy before complex motion. Give the corrected model file a new version; visual resemblance to the robot does not compensate for a failed check.

F2 - Unstable contact. Increase the step, reduce friction or change the foot contact geometry. Expect to detect penetration, sliding or energy growth; repeating with a smaller step and the original geometry should explain the source. Record the physics engine and parameters, since the same SDF can behave differently in another implementation.

F3 - Delayed IMU data and an empty depth map. The sensor supplies old timestamps or rendering fails to produce a valid map. The controller or test must detect stale or missing data; ground truth must never silently replace readings. Specify the frequency of valid messages and the fraction of missing or non-numeric values; explain the total used as the denominator.

Execution and the hardware boundary

F4 - Loss of reproducibility. Repeat one seed after changing the thread count or platform. Compare defined metrics and tolerances rather than claiming unconditional bitwise equality. If contact events differ, isolate a minimal scenario and state which conclusions are materially affected by that instability.

F5 - Data corruption or loss of physical communication. Drop a packet in the HIL stream, alter its CRC or disconnect the line using the method specified in the test. An invalid command must be rejected and data age must increase; the hardware controller and host must record a consistent cause and transition to fallback mode. Specify which side checks CRC and which side decides what to do if communication fails in one direction.

F6 - The simulator cannot keep up. Add computational load or a sensor and measure real-time factor together with data age and deadline compliance. Expect failure of the timing criterion to be recorded even with a normal average RTF. A powerful computer and RTF≈1 do not guarantee freedom from isolated long delays.

Decide how to address each failure

Distinguish a model defect, an interface defect, an environment limitation and missing physical data. Change one factor when correcting a failure. Repeat the reference scenario and preserve the original failure as the basis of a regression test. If a discrepancy remains unexplained, add it to the unresolved fidelity issues with a specific measurement that will distinguish possible causes. "Simulation is imperfect" is too general to support an engineering decision.

Hours 1-40: model physics and sensors

Hours 1-20: simulation foundations

Theory 1-10: 1-2 - SDF structure and the selected engine; 3-4 - mass, inertia and integration; 5-6 - contact and friction; 7-8 - joint limits, damping and actuator model; 9-10 - analytical reference cases and convergence criteria. End each of the five two-hour sessions with a prediction of a specific physical quantity.

Practice 11-20: 11-12 - run a minimal Harmonic world; 13-14 - complete Task 1; 15-16 - repeat with two steps; 17-18 - complete Task 2; 19-20 - save the contact failure and reference configuration. Do not spend the ten-hour practical building a complex robot before the simple cases pass.

Progression criteria: free-fall and pendulum results agree with the chosen analytical calculation, contact has numerical criteria, and the physics step is selected from changes in results and computational cost.

Hours 21-40: model and sensors

Theory 21-30: 21-22 - Xacro and mirroring; 23-24 - inertial/collision and transmissions; 25-26 - gz_ros2_control interfaces; 27-28 - IMU, contacts, depth and ground truth; 29-30 - operation without a GUI, component readiness and time bases. Before installing additional packages, establish that a specific sensor needs them and that the selected environment supports them.

Practice 31-40: 31-32 - verify one leg; 33-34 - complete Task 3 with four legs; 35-36 - connect the sensors; 37-38 - complete Task 4 without a GUI; 39-40 - test an incorrect axis and a missing depth map, and record the manifest. If graphics are unavailable, retain working physics and IMU tests and separately mark depth verification as incomplete.

Progression criteria: the quadruped exists as a verifiable model, joint names match the ROS stack, ground truth is separated from sensors, and missing data is detected explicitly.

Hours 41-75: SIL, PIL, HIL and fidelity

Hours 41-60: the controller inside the loop

Theory 41-50: 41-42 - SIL, PIL and HIL boundaries; 43-44 - state reset and seed; 45-46 - simulation time, wall time and RTF; 47-48 - target numerical types, protocol and seq; 49-50 - timing measurements without mixing clocks. Draw the path of one state snapshot and command, identifying the component responsible for each state.

Practice 51-60: 51-52 - run SIL from Task 5; 53-54 - repeat the experiment and add delay and friction; 55-56 - build the target computational core; 57-58 - run PIL from Task 6; 59-60 - compare host and target calculations, and separately measure computation and round-trip times. Use the verified control law with its previous settings.

Progression criteria: SIL and PIL are reported separately, numerical differences are explained and exchanged data is recorded. Stepwise execution is not presented as a real-time test.

Hours 61-75: the hardware loop and model fidelity

Analysis 61-65: 61-62 - HIL requirements, watchdog and response deadline; 63-64 - comparison of simulation with measurements and subsystem tolerances; 65 - final readiness table. The last theory block consists of 2 + 2 + 1 hours. Keep prepared scenarios within the rig's already verified safe limits.

Practice 66-75: 66-67 - close the Task 7 HIL loop on a real target processor; 68-69 - test physical communication, delay and disconnection; 70-71 - complete Task 8 on an available secured rig or record that physical verification was not completed; 72-73 - prepare the fidelity report and check faults; 74-75 - reproduce the project from a clean start and prepare the handover. Another host-computer run does not replace missing HIL and does not confer the same readiness status.

Progression criteria: the report gives separate statuses for the model, PIL, HIL and physical comparison. Close the open G2 HIL criterion only with the corresponding hardware experiment; subsequent models inherit the measured limitations.

Acceptance checks, limitations and further work

Demonstrate model suitability

Begin the demonstration with an analytical test and a leg-coordinate check, then show the sensor reference world without a GUI. Next, demonstrate one SIL scenario, PIL computation on the target processor and HIL with a physical communication channel. Identify the real and modelled robot components in each experiment. Finish by comparing a physical recording with the model and listing properties that this twin cannot yet predict.

Acceptance requires geometry, inertia and limit checks; recorded contact and timing violations; separate sensor readings and ground truth; a GUI-free test with a timeout; a manifest; a host-versus-target comparison; an HIL log; and unresolved fidelity issues. If hardware is unavailable and HIL or physical comparison has not been completed, record partial readiness and the specific open item. A hil directory or a connected USB cable does not itself establish that the criterion is met.

Self-check questions

1) How does visual differ from collision and inertial? 2) How can you tell whether the physics step is small enough? 3) Why does identical SDF not guarantee identical contact in a different engine? 4) How do you check the axis of a mirrored leg? 5) How does sensor noise differ from delay? 6) Why must ground-truth state be withheld from the operational state estimator?

7) How does a server without a GUI differ from camera rendering without a window? 8) Where does the controller run in SIL, PIL and HIL? 9) Why does RTF≈1 not establish compliance with every deadline? 10) How can you measure communication delay without synchronised clocks? 11) What does HIL with a modelled actuator verify, and what does it leave unverified? 12) Why should digital-twin fidelity be described by subsystem? Answer using your own tests and hardware boundaries.

Handover and selection of physical experiments

Pass the worlds, sensor models, actuator settings, scenario catalogue and list of permitted uses to later stages. For example: the model is suitable for interface and delay testing, has limited suitability for tuning small movements and is not yet validated for impact contact. Plan separate tests of heating, backlash, foot forces, friction on the actual surface and emergency-response delay; these cannot be inferred solely from a gait simulation.

If time is short, reduce visual mesh detail, the number of worlds and use of the additional MuJoCo engine. Retain the four-leg structure, one thoroughly verified dynamic leg, sensor reference scenes and the hardware loop. Do not add a new gait algorithm, RL or a complete arm model to these 75 hours. Turn model errors into specific engineering tasks with an expected measurement, rather than a general doubt about the simulator's usefulness.

Assigned reading: a focused route through the sections

The literature is mainly in English; the programme and definitions are also in English. Read the selected sections before the relevant task, rather than entire books. Core reading is included in the existing 35 analysis hours; consult references as needed.

Core reading

E. Renard, ROS 2 from Scratch (2024). Book and contents. Ch. 13 Simulating a Robot in Gazebo: How Gazebo works, Adapting the URDF for Gazebo, Spawning the robot in Gazebo. Before Tasks 3-4, understand the relationship between model, world and plugin. Use commands appropriate to Jazzy/Harmonic.

K. Lynch, F. Park, Modern Robotics (2017). Learning resources. §8.1.3 Understanding the Mass Matrix, §8.2 Dynamics of a Single Rigid Body and §8.5 Forward Dynamics of Open Chains. Before Tasks 1 and 8, check inertia, the justification for the pendulum reference case and the meaning of forward dynamics.

R. Tedrake, Underactuated Robotics, MIT notes. Multi-Body Dynamics. Sections The Dynamics of Contact and Compliant Contact Models. Before Task 2, establish the relationship between penetration, stiffness, friction and integration step; calculate the condition for the onset of sliding.

Implementation references

SDFormat specification - model/link/inertial, joint/axis, world/physics and sensor. In Tasks 1-4, record the schema version and check which fields the selected engine uses.

Gazebo Harmonic: Sensors - IMU sensor and Contact sensor. For Task 4, study frequency, coordinate frames, plugins, noise and separation of sensor readings from simulation ground truth.

gz_ros2_control Jazzy - Add ros2_control tag, demos and Notes on the command interface. In Tasks 3 and 5, check which quantity the interface commands: torque, velocity or position.

What to record in the laboratory log

Give a source for every parameter: CAD, specifications, measurement or an assumption with a range. After reading, identify one numerical physics test and one boundary it does not establish. HIL requires a real controller and channel; conclusions about mechanics require a separate physical experiment.

Terms and definitions

Digital twin. A model of a specific system tied to its geometry, parameters and measurements within a declared domain of applicability.

SDF. A format describing simulation worlds, bodies, joints, physics and sensors; tag semantics depend on the supported schema.

Inertial parameters. A body's mass, centre of mass and inertia tensor, expressed relative to a specified coordinate frame.

Collision geometry. Geometry used to detect collisions and contacts; it may differ from the model's detailed visual surface.

Forward dynamics. Calculation of accelerations from state, forces and constraints; an integrator then updates the state over time.

Integration step. The increment of simulation time between numerical updates; it affects error, stability and computational cost.

Step-size convergence. Stabilisation of a selected result as the step is progressively reduced; checked with identical conditions and duration.

Contact. Interaction between surfaces that come together, involving normal and possibly tangential forces; contact can allow sliding.

Normal force. The contact-force component perpendicular to the surface; in a unilateral contact model, the surface pushes but does not pull.

Friction coefficient μ. The dimensionless ratio of limiting tangential force to normal force in the adopted model; it depends on materials and contact regime.

Penetration. Calculated overlap of contacting surfaces; in a compliant model it determines force together with stiffness and damping.

Compliant contact. A model in which force depends on deformation and its rate; high stiffness places stronger demands on the integrator.

Ground-truth state. The model's true internal state, used to calculate errors; the operational state estimator must not receive it.

Seed. The initial state of a random-number generator; it fixes the random sequence, but not every effect of parallel execution.

SIL. A closed-loop software experiment in which a host-computer controller acts on a model and receives its response.

PIL. An experiment using controller code on the target processor and a plant model; coordinated stepwise execution is permitted.

HIL. A closed-loop experiment with a real controller and channel, checking explicitly declared hardware boundaries and timing requirements.

Real-time factor. The ratio of simulation-time and wall-time increments; it does not describe command latency.

Round-trip time. The time for an outward-and-return exchange measured using one clock; without further assumptions it does not give one-way delay.

Model validation. Checking predictions against independent measurements for a specified task; it does not establish applicability in every operating regime.

Equipment and setup for this month

Use existing equipment

MacBook Pro M4, Ubuntu 24.04 ARM64 in UTM and the Jazzy environment from M12; ESP32-S3, a USB/UART/CAN channel, analyser and power supply from earlier modules. HIL requires a real microcontroller (MCU) that calculates control, and a real channel. The M7 bench motor remains suitable for repeating joint measurements.

Purchase after verifying the selected actuator

A separate physical 3DOF leg requires compatible actuators from the selected family. If one actuator has already passed torque, speed, heating, feedback, protocol and shutdown checks, buy another 2, making 3 in total. If there is no verified candidate, first buy and test 1; order the next 2 only after it passes acceptance. The bench DC motor does not automatically count as that candidate.

For the approved leg CAD model and BOM, prepare 1 set of printed links, fasteners, bearings and wiring to the final geometry, plus 1 restraining fixture. Use the existing 3D printer with a profile for its exact model. Do not yet buy complete sets of 12/18 actuators or an onboard Pi/x86 computer. Single-leg readiness is established by measurements, not the G2 date.

Install now

In the same Ubuntu guest, install Gazebo Harmonic, ros_gz/bridge, gz_ros2_control and the demos. Follow the official ROS/Gazebo pairing and Jazzy plugin installation; use ros-jazzy-gz-ros2-control. Do not mix Harmonic with Gazebo Classic. Retain ESP-IDF, firmware and the protocol from M6-M12.

Check the installation and its limitations

Run the cart demo, check joint states and position changes, then verify free fall and the pendulum with dt and dt/2. Separately check depth-map rendering without a GUI: a server without a window does not establish that depth frames exist. In UTM, check USB exchange and measure dropouts and round-trip time; if timing cannot be maintained, leave the HIL criterion open.

Simple HIL with a microcontroller and model does not require a physical leg, but it does not validate mechanics either. Purchasing three actuators serves the separate physical experiment in Task 8. Preparation fits within the 75 hours in place of repeated setup; delivery time is outside study hours. Power on new actuators only after matching the supply, limits and mounting.

Motion fundamentals: contact and model fidelity

Required: why touching the ground does not ensure support

A foot can touch the floor while sliding, and a contact that appears rigid can still allow penetration. Before moving on to the leg, check these effects with a simple body. Before calculating, resolve gravity into normal and tangential components and explain why static friction need not equal μFn.

A minimal friction model on an incline

A block of m = 1 kg neither rotates nor tips. The plane is stationary at angle α; the local t-axis points downhill and n points outwards. g = 9.81 m/s². Fn ≥ 0 and Ft are the forces exerted by the plane on the block, in N; acceleration along n is zero: **Fn = mg cos α; m at = mg sin α + Ft**. Friction coefficients are dimensionless.

Rest requires Ft = −mg sin α and **|Ft| ≤ μs Fn**, where μs is the coefficient of static friction. During downhill sliding, **Ft = −μk Fn**, where μk is the coefficient of kinetic friction. The normal force does not oppose motion along the slope; the boundary of static equilibrium here is tan α = μs.

Let μs = 0.40 and μk = 0.30. At α = 20°: Fn = 9.218 N, mg sin α = 3.355 N, and the friction limit is 3.687 N. Thus Ft = −3.355 N is admissible, and a block initially at rest remains stationary. The actual friction is smaller than its limiting value.

At α = 30°: Fn = 8.496 N, the downhill force is 4.905 N, and the limiting static-friction force is 3.398 N. Rest is impossible; during sliding, Ft = −2.549 N and **at = 9.81(sin 30° − 0.30 cos 30°) = 2.356 m/s²**. Friction dissipates mechanical energy: Ft·vt ≤ 0. The wrong sign would cause energy to grow.

In the spring model, Fn = kδ, where k is stiffness in N/m and δ is penetration in m. At rest on a 20° incline with k = 10 000 N/m: δ = 9.218/10 000 = **0.000922 m, or 0.922 mm**. This is consistent with the adopted compliance; a zero visible gap is not an accuracy criterion.

Increasing k changes the physical model and raises the oscillation frequency; reducing the step Δt changes numerical accuracy. Compare Δt and Δt/2 with identical mass, friction, stiffness and solver. If the engine uses a different contact law, first map its parameters: δ = Fn/k may not apply.

Two experiments within existing Tasks 1-2

1. Start the block from rest at 20° and 30°; save forces, velocity and acceleration after the transient. Verify rest or sliding, the sign of Ft and the numbers above. If the engine uses a single μ, set μs = μk and recalculate the prediction; record the settings explicitly.

2. For 30°, compare two steps over the same interval of established sliding. Save mean at, Fn, penetration, kinetic energy K and potential energy U. The teaching target for at is a difference from the analytical value and between steps of ≤ 2% of 2.356 m/s² with the original μs and μk. Check that K may increase as U decreases; total mechanical energy, including the contact spring, decreases through friction.

Time: 1 theory hour from 5-6 (contact), plus 1 practice hour each from 15-16 (two steps) and 17-18 (Task 2). These calculations and checks are included in the 75 hours. The block validates the selected contact regime; the complete leg and physical HIL require their own checks.

Read: Lynch, Park, Modern Robotics §12.2.1: Friction; Tedrake, Multi-Body Dynamics → Compliant Contact Models.